

FINAL PROJECT REPORT

Development of Indigenous Contactless
Integrated Track Monitoring Systems (ITMS)
for Track Recording

CSE 471 — Deep Learning for Computer Vision

Software-Based Railway Track Inspection Prototype

Dataset: Railway Fastener Defect Dataset (RFDD)
Model: YOLO26n

DEVELOPMENT OF INDIGENOUS CONTACTLESS INTEGRATED TRACK MONITORING SYSTEMS (ITMS) FOR TRACK RECORDING

Chapter 1 — Introduction, Background, Problem Statement, Objectives and Scope

Project domain	Deep Learning / Computer Vision
Prototype focus	Railway fastener inspection and track recording
Core model	YOLO26n
Dataset	RFDD — Railway Fastener Defect Dataset
Output	Automated inspection records, annotated video, HTML report and ITMS dashboard

Purpose of this chapter: Establish the railway-inspection context, define the problem addressed by the project, state the objectives, and clearly delimit the software prototype so that its contribution is technically accurate and academically defensible.

1.1 Background

Railway tracks are safety-critical infrastructure whose condition must be monitored continuously and systematically. Inspection activities traditionally involve measurements, visual examination and maintenance processes that must identify defects early enough for corrective action. As railway networks adopt more advanced monitoring technologies, computer vision, machine learning and automated data processing can support the inspection workflow by converting visual track information into structured condition records.

Indian Railways has adopted Integrated Track Monitoring Systems (ITMS) for comprehensive track inspection and monitoring. A March 2026 Ministry of Railways/Press Information Bureau release states that ITMS uses machine learning and image processing to monitor and detect defects in railway track components such as rails, sleepers and fastenings. The same release reports that three ITMS are deployed for track recording and monitoring. [1]

An earlier official Ministry of Railways release describes ITMS installed on Track Recording Cars and notes the use of contactless monitoring technologies including laser sensors, high-speed cameras and LiDAR. It also describes video inspection and machine learning for identifying structural defects and integration with the Track Management System for reporting. [2]

These real-world systems establish the motivation for this academic prototype: demonstrate how a computer-vision pipeline can automatically inspect visible railway components, identify their condition and convert the results into useful inspection records.

1.2 Indian Railways ITMS Context

Real-world ITMS aspect	Relevance to this academic prototype
Contactless inspection	The project uses image/video-based analysis rather than physical contact sensors.
High-speed cameras / video inspection	The prototype processes railway images and a simulated inspection video.
Machine learning and image processing	YOLO26n performs object detection and condition classification.
Track-component defect detection	The prototype concentrates on railway fastener conditions.
Inspection data and reporting	Predictions are converted into CSV records, an HTML report and a dashboard.

The project intentionally mirrors the software/data-processing portion of the ITMS concept. It does not attempt to reproduce every subsystem of a production Track Recording Car.

1.3 Problem Statement

Railway fasteners are important visible components of track infrastructure. Their condition can vary across normal and defective states, including deformation, fracture, missing fasteners, inversion and displacement. Manual examination of large quantities of visual inspection data can be time-consuming and may make consistent recording difficult.

Problem statement: Develop a computer-vision-based software prototype inspired by an Integrated Track Monitoring System that can automatically detect railway fasteners from inspection images/video, classify their condition, record the inspection results and present the outputs through an accessible reporting/dashboard interface.

1.4 Motivation

- Reduce the amount of manual visual screening required for large collections of inspection imagery.
- Provide consistent machine-generated labels and confidence scores for visible fastener conditions.
- Convert raw image/video predictions into structured inspection records that are easier to review.
- Demonstrate the application of deep learning for a safety-oriented railway computer-vision problem.
- Build a software prototype that reflects the inspection-and-reporting workflow of modern contactless track monitoring.

1.5 Aim of the Project

The primary aim is to develop an AI-based computer-vision prototype for contactless railway fastener inspection and automated track recording. The system is designed to accept railway imagery, detect fasteners, classify their visible condition, generate inspection records and provide visual reporting through a dashboard.

1.6 Objectives

1. Study the concept and software-oriented workflow of Integrated Track Monitoring Systems.
2. Prepare a railway fastener image dataset for deep-learning-based object detection.
3. Train a YOLO26n model to detect and classify six railway fastener conditions.
4. Evaluate the trained model using an official held-out RFDD test set.
5. Apply the model to frame/image sequences and a simulated inspection video.
6. Automatically generate structured inspection records containing predicted class and confidence information.
7. Produce annotated inspection images and video for visual verification.
8. Create an HTML inspection report and ITMS dashboard for summarizing inspection results.
9. Document the system's performance, limitations and possible future extensions.

1.7 Scope of the Project

Included in scope

- Railway fastener detection using RGB imagery.
- Six-class fastener condition classification: Deformed, Fractured, Missing, Inverted, Normal and Displaced.
- YOLO26n training and evaluation.
- Image-based and video-based inference.
- Automatic recording of predictions.
- Annotated inspection outputs.
- HTML report and standalone dashboard visualization.

Outside the present scope

- Physical Track Recording Car construction or modification.
- LiDAR, laser, accelerometer or other industrial sensor integration.
- Engineering-grade measurement of track geometry, rail profile, rail wear or ride quality.
- Direct replacement of certified railway inspection procedures.
- Real railway deployment or safety certification.
- Claiming that demonstration-video prediction counts are ground-truth defect counts.

1.8 Significance of the Project

The significance of the project lies in demonstrating a complete computer-vision inspection workflow rather than only training an isolated object-detection model. The prototype connects dataset preparation, deep-learning inference, condition classification, structured recording, video inspection and visualization. This makes the project relevant to both deep-learning/computer-vision study and the broader concept of intelligent railway asset monitoring.

1.9 Expected Outcome

Input	Processing	Output
Railway images / simulated video	YOLO26n detection and classification	Fastener class + confidence + bounding box
Model predictions	Automated recording	CSV inspection records
Annotated frames	Report generation	HTML inspection report
Inspection video + statistics	Visualization	ITMS dashboard

1.10 Chapter Summary

This chapter established the railway-inspection context and defined the project as a software-based, computer-vision prototype inspired by Indian Railways ITMS. The project focuses specifically on automated railway fastener inspection, classification and recording using deep learning. The next chapters will describe the existing/related systems, dataset and methodology, implementation, experimental results and conclusions.

References Used for the ITMS Background

[1] Press Information Bureau, Ministry of Railways, "Indian Railways Deploys Advance AI & Machine Learning Devices to Enhance Safety and its Operational Efficiency by Adopting Smart Monitoring," 12 March 2026.

[2] Press Information Bureau, Ministry of Railways, "High-tech rail cum road inspection vehicle and state of art rail track health monitoring to ensure safe train travel for Indians," 5 December 2024.

[3] Research Designs and Standards Organisation (RDSO), Indian Railways, "Indian Railways Annual Report and Accounts 2022-23," section on Integrated Track Monitoring System.

[4] RDSO, Indian Railways, "Draft Technical Specification for Integrated Track Monitoring System from Zero Speed," Specification No. TM/IM/448.

Official source pages: pib.gov.in; rdso.indianrailways.gov.in. Accessed for this chapter in September 2026.

DEVELOPMENT OF INDIGENOUS CONTACTLESS INTEGRATED TRACK MONITORING SYSTEMS (ITMS) FOR TRACK RECORDING

Chapter 2 — Literature Review and Existing Approaches

Purpose: Review the technological approaches relevant to automated railway track inspection, railway fastener defect detection, deep-learning object detection and the ITMS concept, then identify the gap addressed by the proposed prototype.

2.1 Overview of Railway Track Inspection

Railway track inspection has progressively moved from predominantly manual examination toward instrumented and automated monitoring. Modern inspection workflows can combine imaging, sensors, measurement systems and software analytics to identify components that require attention. The objective is not simply to collect images but to transform inspection observations into information that can support maintenance decisions.

Indian Railways' current ITMS deployment provides a relevant real-world context. The Ministry of Railways states that ITMS uses machine learning and image processing to monitor and detect defects in rails, sleepers and fastenings, and that ITMS data are analyzed for urgent and planned track maintenance. [1]

A 2024 Ministry of Railways release describes ITMS installed on Track Recording Cars and reports contactless monitoring using laser sensors, high-speed cameras and LiDAR. It also describes video inspection, machine learning and integration with the Track Management System for reporting. [2]

2.2 Conventional / Manual Inspection Approach

- Human inspectors visually examine track components and identify abnormal conditions.
- Measurements and observations may be recorded for subsequent maintenance planning.
- Manual inspection can provide expert judgement, but large-scale visual review can be time-consuming.
- Consistency can be affected by fatigue, environmental conditions and the difficulty of reviewing large volumes of imagery.

The proposed project does not attempt to eliminate human expertise. Instead, it demonstrates an automated screening layer that can identify candidate fastener conditions and produce structured records for subsequent review.

2.3 Machine-Vision-Based Railway Inspection

Machine vision uses cameras and image-processing algorithms to inspect visible infrastructure. In railway applications, visual inspection can target rails, sleepers, fasteners and other components. The key advantage is that the same visual input can be retained as evidence while automated algorithms process large numbers of frames.

The current ITMS context is particularly relevant because Indian Railways explicitly identifies machine learning and image processing as technologies used for track-component defect detection. Therefore, a deep-learning prototype focused on fasteners is aligned with the software-analysis portion of the broader ITMS workflow, while remaining much smaller in scope than a production railway inspection system. [1][2]

2.4 Traditional Image-Processing Methods

Before the widespread use of deep learning, computer-vision inspection systems commonly relied on manually designed image-processing operations such as thresholding, edge detection, contour extraction, template matching and handcrafted feature descriptors. These techniques can be effective when object appearance, illumination and camera geometry are tightly controlled.

However, railway inspection imagery can contain changes in illumination, perspective, scale, texture, occlusion and background structure. Designing a separate set of robust rules for every defect condition can therefore become difficult. Deep-learning approaches address this by learning visual representations directly

from annotated examples.

2.5 Deep Learning for Object Detection

Deep learning enables a model to learn discriminative visual features from training data rather than relying entirely on manually designed image features. For railway fastener inspection, this is useful because the system must distinguish between multiple visually related conditions while also locating the fastener within a full-scene image.

Object detection combines two tasks: localization of an object using a bounding box and classification of the detected object. This makes object detection appropriate for the present project because each fastener must be located and assigned one of six condition classes.

2.6 YOLO-Based Object Detection

The YOLO (You Only Look Once) approach formulated object detection as a unified prediction problem in which a neural network directly predicts bounding boxes and class probabilities from an image. The original YOLO work emphasized a single end-to-end network and high processing speed, making the family influential for real-time object detection. [3]

Subsequent YOLO generations introduced architectural and training improvements while retaining the general objective of balancing detection accuracy and computational efficiency. This balance makes YOLO-style detectors attractive for applications in which inspection imagery may arrive as a continuous stream.

For this project, YOLO26n was selected as the final model. The choice supports the project's requirement for a compact object detector capable of processing full-scene railway imagery while producing bounding boxes and class predictions.

2.7 Railway Fastener Defect Detection

Railway fasteners are important track components because they secure rails to sleepers or slabs and contribute to maintaining the required track configuration. Defects can include missing, inverted, displaced, deformed and fractured conditions. These conditions can be visually subtle, making representative training data important for reliable automated inspection.

The 2026 RFDD Scientific Data publication introduced a high-resolution railway fastener benchmark specifically designed for intelligent inspection of high-speed railway infrastructure. RFDD contains 1,350 full-scene images at 2048 × 2021 resolution and more than 8,100 annotated fastener instances, with one normal class and five defect categories: Missing, Inverted, Displaced, Deformed and Fractured. [4]

An important characteristic of RFDD is that it preserves full-scene railway layouts rather than relying only on cropped single-fastener images. The dataset was constructed with engineering-oriented consistency principles and an image-level split intended to reduce leakage between training, validation and testing. [4]

2.8 Comparison of Relevant Approaches

Approach	Strength	Limitation / consideration
Manual visual inspection	Uses human expertise and contextual judgement	Time-consuming for large image/video volumes; recording consistency
Traditional image processing	Simple and interpretable under controlled conditions	Rules/features may be sensitive to illumination, scale and appearance
General deep-learning detection	Learn visual features and localizes objects automatically	Requires representative labeled data and computational resources
YOLO-style detection	Combines localization and classification with strong speed/accuracy trade-off	Performance depends on training data, model size and scene conditions

Approach	Strength	Limitation / consideration
Proposed ITMS prototype	Connects detection, classification, recording, video inspection and dashboards	Limited for standardised reporting; dataset-based demonstration; not a

2.9 Research / Project Gap

The reviewed approaches reveal a gap between isolated object-detection experiments and an end-to-end inspection workflow. A model may successfully detect a fastener, but an inspection-oriented system also needs to organize predictions, retain evidence, generate records and communicate results.

- Many computer-vision demonstrations stop at bounding-box predictions rather than producing an inspection record.
- A railway-specific system must handle multiple condition classes rather than only detecting whether a fastener exists.
- Practical inspection workflows require outputs that can be reviewed, summarized and reported.
- The project therefore combines deep-learning detection with automated recording, annotated video, HTML reporting and dashboard visualization.

2.10 Positioning of the Proposed Project

The proposed project should be positioned as a **software-based ITMS-inspired computer-vision prototype**. Its contribution is not the recreation of every hardware and measurement subsystem used by Indian Railways. Instead, it demonstrates a focused pipeline for visible fastener inspection: image/video input → YOLO26n detection → six-class condition classification → structured recording → visual reporting.

Real ITMS context	This project
Track Recording Car	Laptop/PC software prototype
Cameras + LiDAR + laser sensors	Image/video input for computer vision
Multiple track parameters	Fastener-condition detection
Machine learning + image processing	YOLO26n deep-learning inference
Track Management System integration	CSV + HTML report + dashboard
Operational railway deployment	Dataset/video demonstration

2.11 Chapter Summary

The literature and technology review shows a progression from manual inspection and handcrafted image processing toward machine-vision and deep-learning-based inspection. Indian Railways' current ITMS context confirms the relevance of machine learning, image processing and contactless visual monitoring to railway track-component inspection. The RFDD dataset provides a railway-specific full-scene benchmark for fastener defect detection, while YOLO-style detectors provide an appropriate framework for simultaneous localization and classification. The proposed project combines these elements into an end-to-end software prototype that emphasizes not only detection but also recording and reporting.

References

[1] Press Information Bureau, Ministry of Railways, Government of India. “Indian Railways Deploys Advance AI & Machine Learning Devices to Enhance Safety and its Operational Efficiency by Adopting Smart Monitoring.” 12 March 2026.

[2] Press Information Bureau, Ministry of Railways, Government of India. “High-tech rail cum road inspection vehicle and state of art rail track health monitoring to ensure safe rail travel for Indians.” 5 December 2024.

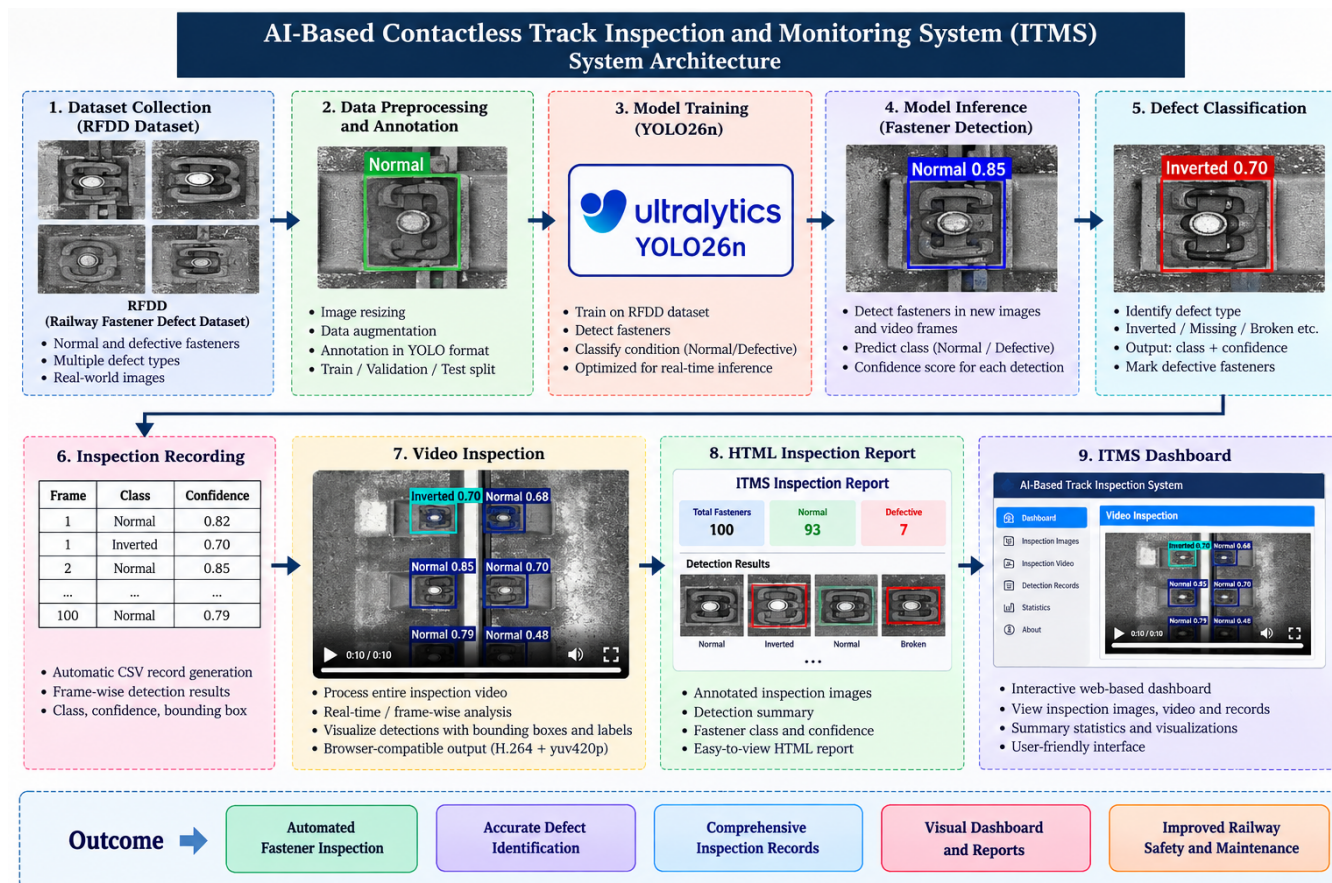
[3] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. “You Only Look Once: Unified, Real-Time Object Detection.” arXiv:1506.02640, 2015.

[4] Wang, B., Pei, C., Xiong, X., et al. "RFDD: A High-Quality Dataset based on Metrology for Fastener Defect Detection in High-Speed Railways." *Scientific Data* 13, 1178 (2026). DOI: 10.1038/s41597-026-07851-7.

Source pages used for verification: Press Information Bureau (pib.gov.in), arXiv, and Nature Scientific Data.

DEVELOPMENT OF INDIGENOUS CONTACTLESS INTEGRATED TRACK MONITORING SYSTEMS (ITMS) FOR TRACK RECORDING

Chapter 3 — Proposed Methodology and System Design



This chapter defines the technical methodology implemented in the project, from railway image input through deep-learning inference and automated inspection reporting.

3.1 Proposed System Overview

The proposed system is a software-based computer-vision prototype inspired by the contactless inspection workflow of an Integrated Track Monitoring System. The system accepts railway fastener imagery or a sequence of inspection frames, processes the input using a trained YOLO26n detector, identifies fasteners and their visible condition, and converts the predictions into structured inspection outputs.

The methodology deliberately separates model evaluation from demonstration outputs. Model quality is measured on the official RFDD test set using precision, recall, mAP@0.50 and mAP@0.50:0.95. The dashboard and video counts are treated as inference outputs rather than accuracy measurements.

3.2 System Architecture

Layer	Function
Input layer	Railway fastener images or simulated inspection video frames
Data layer	RFDD dataset, labels and official train/validation/test organization
Model layer	YOLO26n object detector trained for six fastener-condition classes
Inference layer	Bounding-box detection, class prediction and confidence estimation
Recording layer	Frame/image identifiers, predicted class, confidence and bounding-box information
Reporting layer	Annotated images, CSV inspection records and HTML inspection report
Visualization layer	Standalone ITMS dashboard and browser-compatible inspection video

3.3 Data Acquisition and Preparation

The RFDD dataset is provided in a high-resolution HDF5-based training/validation representation and a PNG/TXT-based test representation. The training/validation HDF5 image collection contains duplicate image entries because each unique image is represented twice in the supplied image dataset. The project therefore retained one copy of each unique image name and matched it with its corresponding label.

The official image-level split was preserved: 1,050 training images, 200 validation images and 100 test images. The labels were decoded as normalized YOLO bounding-box records in the form **class_id**, **center_x**, **center_y**, **width**, **height**.

3.4 Fastener Condition Classes

ID	Class	Meaning in the prototype
0	Deformed	Fastener shows a deformed condition
1	Fractured	Fastener shows a fractured condition
2	Missing	Expected fastener is missing
3	Inverted	Fastener is in an inverted orientation
4	Normal	Fastener appears normal
5	Displaced	Fastener is displaced from its expected position

3.5 Model Selection

YOLO26n was selected as the final detection model. The project requires simultaneous localization and classification of multiple fasteners in full-scene railway images. A compact YOLO model provides a practical balance between detection capability and computational requirements for a software prototype.

The model predicts a bounding box and class for each detected fastener. Confidence scores are retained so that inspection results can be reviewed rather than treated as unconditional decisions.

3.6 Training Methodology

1. Prepare the RFDD images and labels in YOLO-compatible directory structure.
2. Preserve the official training, validation and test separation.
3. Configure the six fastener-condition class names.
4. Train YOLO26n on the training set while monitoring validation performance.
5. Compare model performance after different training durations.
6. Select the 40-epoch model as the final model because it improved official test performance over the 20-epoch run.
7. Evaluate the selected model on the held-out official test set.

3.7 Training Configuration Used

Parameter	Project setting
Model	YOLO26n
Dataset	RFDD
Training duration	40 epochs
Validation	Official 200-image validation split
Final evaluation	Official 100-image test split
Execution environment	Windows PC; CPU-based PyTorch in current setup
Output	Best trained weights (best.pt)

3.8 Image Inference Workflow

- Load the trained YOLO26n weights.
- Read an inspection image.
- Run object detection.
- For every prediction, obtain bounding box, class and confidence.
- Draw the detection on the image for visual verification.
- Write the prediction to the inspection CSV.
- Aggregate counts for normal and defective conditions.
- Store annotated output images.

3.9 Video Inspection Workflow

For video demonstration, the inspection video is processed frame by frame. Each frame is passed through the trained model, predictions are drawn onto the frame, and the resulting annotated frame is written into an output video. At the same time, predictions are recorded in a CSV file for later inspection and aggregation.

Step	Operation
1	Open inspection video and read video properties such as frame rate and resolution.
2	Read the next frame.
3	Run YOLO26n inference.
4	Extract detections and confidence scores.
5	Annotate the frame with bounding boxes and class labels.
6	Record prediction information in CSV.
7	Write annotated frame to output video.
8	Repeat until all frames are processed.
9	Aggregate total normal and defective predictions for the dashboard.

3.10 Inspection Recording Design

The recording subsystem converts predictions into machine-readable inspection data. A record is generated for each detected fastener, allowing the inspection output to be searched, counted and summarized later.

Field	Purpose
Frame / image	Identifies where the detection occurred
Class	Predicted fastener condition
Confidence	Model confidence for the prediction
Bounding box	Location and extent of the detected fastener

3.11 Defect Aggregation

The five non-normal classes are treated as defective conditions for the prototype's summary view. The Normal class is kept separate. This allows the dashboard to display total detections, normal detections, defective detections and a predicted defect ratio.

Predicted defect ratio = (number of predicted defective fasteners / total predicted fastener detections) × 100.

3.12 Reporting and Visualization

Output	Purpose
Annotated images	Visual verification of model predictions
Inspection CSV	Structured machine-readable inspection record
Annotated video	Frame-by-frame demonstration of automated inspection
HTML inspection report	Summary of inspection statistics and metadata
ITMS dashboard	Interactive visual presentation of video results and inspection information

3.13 Browser-Compatible Video Processing

The initial annotated MP4 used a codec configuration that was not decoded correctly by the browser when embedded in the local HTML dashboard. FFmpeg was subsequently used to create a browser-compatible H.264 video with yuv420p pixel format and fast-start metadata. This allowed the inspection video to play directly inside the dashboard.

3.14 End-to-End Algorithm

1. Start.
2. Load trained YOLO26n model.
3. Load image or inspection video.
4. For each image/frame, perform object detection.
5. For every detection, obtain class, confidence and bounding box.
6. Store prediction and create annotated visual output.
7. Classify predictions as Normal or Defective for summary statistics.
8. Write structured inspection record.
9. For video, continue until all frames are processed.
10. Generate report and dashboard statistics.
11. Display inspection results.
12. End.

3.15 Methodological Design Principles

- Use an official held-out test set for final model evaluation.
- Keep model metrics separate from demonstration-video prediction counts.
- Preserve the original dataset split to reduce evaluation leakage.
- Retain confidence and localization information rather than reporting only class names.
- Provide visual evidence through annotated images and video.
- Clearly state that the prototype is experimental and not an engineering-certified railway inspection system.

3.16 Chapter Summary

This chapter defined the proposed ITMS-inspired computer-vision methodology. The system combines RFDD data preparation, YOLO26n training, image and video inference, automated inspection recording, visual annotation, report generation and dashboard visualization. The design provides an end-to-end software pipeline while deliberately limiting claims to the capabilities demonstrated by the available dataset and computer-vision model.

DEVELOPMENT OF INDIGENOUS CONTACTLESS INTEGRATED TRACK MONITORING SYSTEMS (ITMS) FOR TRACK RECORDING

Chapter 4 — Implementation and Development

This chapter documents the actual software implementation completed for the computer-vision ITMS prototype, including the project structure, dataset conversion, model training, inference, recording, video processing and dashboard generation.

4.1 Development Environment

Component	Implementation
Operating system	Windows 10/11 environment
Programming language	Python 3.13.7
Deep-learning framework	PyTorch
Object-detection framework	Ultralytics
Final model	YOLO26n
Computer vision / video	OpenCV
Dataset processing	h5py, Python data-processing scripts
Reporting	HTML/CSS generated locally
Video conversion	FFmpeg 9.0.1
Execution for final training	CPU-based PyTorch setup

4.2 Project Directory Structure

```
C:\downloads\RFDD_YOLO
■■■ images
■   ■■■ train
■   ■■■ val
■   ■■■ test
■■■ labels
■   ■■■ train
■   ■■■ val
■   ■■■ test
■■■ itms_track_recording.py
■■■ create_test_video.py
■■■ itms_video_monitor.py
■■■ ITMS_Output
    ■■■ track_inspection_record.csv
    ■■■ annotated_images
    ■■■ simulated_track_inspection.mp4
    ■■■ ITMS_Inspection_Report.html
    ■■■ ITMS_Dashboard.html
    ■■■ video_inspection
        ■■■ video_inspection_record.csv
        ■■■ ITMS_inspection_result.mp4
        ■■■ ITMS_inspection_browser.mp4
```

4.3 Dataset Conversion and Organization

The supplied RFDD training/validation data were stored in HDF5 files. The image dataset contained 2,500 entries representing 1,250 unique image names, while the label dataset contained 1,250 label entries. The duplicate image entries were therefore reduced to one copy per unique image and matched against the corresponding label.

The converted dataset was organized into the standard YOLO directory structure with separate image and label folders for training, validation and testing. The official RFDD split of 1,050 training images, 200 validation images and 100 test images was retained.

4.4 YOLO Label Preparation

Each fastener annotation was represented using normalized YOLO coordinates. The label record contains five values:

```
class_id center_x center_y width height
```

- `class_id` identifies one of the six fastener-condition classes.
- `center_x` and `center_y` specify the normalized center of the bounding box.
- `width` and `height` specify normalized bounding-box dimensions.
- Coordinates are normalized relative to the image width and height.

4.5 Model Training Implementation

The prepared dataset was supplied to the Ultralytics training pipeline using a dataset configuration containing the six class names. YOLO26n was trained for 40 epochs. The best model weights were retained for final inference and evaluation.

An earlier 20-epoch training run was also evaluated. The 40-epoch model produced improved official test performance and was therefore selected as the final model.

Model run	mAP@0.50	mAP@0.50:0.95
20 epochs	90.7%	65.8%
40 epochs — final	94.5%	70.7%

4.6 Model Evaluation Implementation

The final model was evaluated on the official 100-image RFDD test split. The evaluation generated precision, recall, mAP@0.50 and mAP@0.50:0.95 values, together with class-wise performance and a confusion matrix.

Metric	Final result
Precision	0.941
Recall	0.888
mAP@0.50	0.945
mAP@0.50:0.95	0.707

4.7 Image-Based Track Recording Implementation

The track-recording script loads the trained model and processes the official test images. For each detected fastener, the script records the predicted class, confidence and location information and creates an annotated image.

The image-based run processed 100 test images and produced 584 predicted fastener detections. The resulting CSV contains 584 records. The prediction summary was 484 Normal, 19 Deformed, 20 Fractured, 21 Missing, 20 Inverted and 20 Displaced.

Implementation note: These 584 detections and their class counts are inference outputs from the recording script. They are not the official test-set ground-truth counts and must not be used as the model's accuracy.

4.8 Simulated Inspection Video Generation

Because the available RFDD data are still images, a demonstration video was generated by arranging the 100 official test images into a frame sequence. The resulting video was used to demonstrate the temporal inspection workflow without claiming that it represents a live railway camera feed.

Property	Value
Source	100 RFDD official test images
Output resolution	1280 × 720
Frame rate	10 FPS
Purpose	Demonstration of frame-by-frame ITMS inspection

4.9 Video Monitoring Implementation

The video-monitoring script opens the simulated video, reads frames sequentially, performs YOLO26n inference on each frame, draws the detected fasteners and writes the annotated frames to an output video. A separate CSV records the frame-level prediction information.

The completed video run processed all 100 frames and produced 535 predicted fastener detections, including 79 predictions classified as defective and 456 classified as normal.

Video result	Value
Frames processed	100
Fastener detections	535
Normal predictions	456
Defective predictions	79
Predicted defect ratio	14.8%

4.10 Video Codec Compatibility Fix

The first annotated MP4 could not be decoded correctly by the browser when embedded in the dashboard. FFmpeg 9.0.1 was installed and used to convert the video to an H.264 stream with yuv420p pixel format and fast-start metadata.

```
ffmpeg -i "ITMS_inspection_result.mp4" -c:v libx264 -pix_fmt yuv420p -movflags +faststart "ITMS_inspection_browser.mp4"
```

The converted browser-compatible video successfully played inside the local HTML dashboard, resolving the original black-video/0:00 playback problem.

4.11 HTML Inspection Report

A standalone HTML inspection report was generated from the model outputs. It summarizes the inspection statistics, model information, dataset information and the prototype disclaimer.

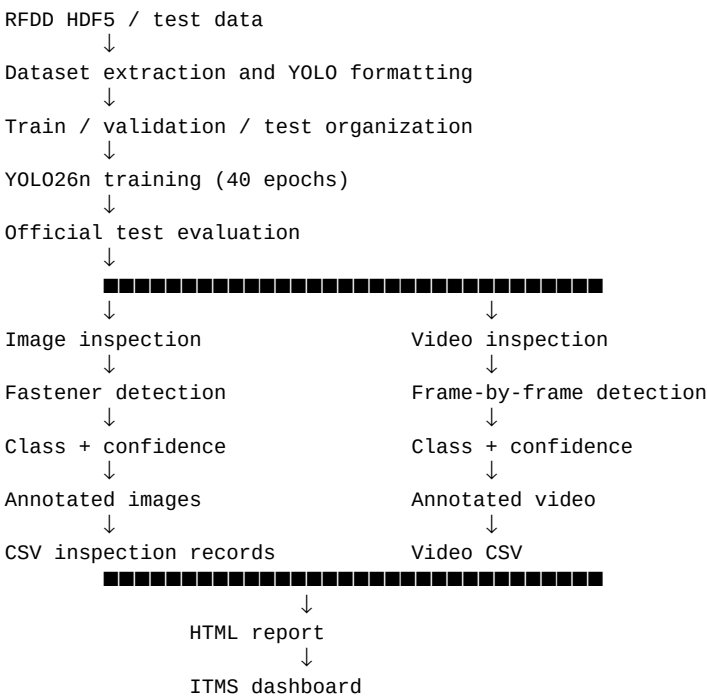
- Total predicted fastener detections.
- Normal and defective prediction counts.
- Defect-class breakdown.
- Model and dataset information.
- Inspection output summary.
- Experimental-prototype safety disclaimer.

4.12 ITMS Dashboard Implementation

The final user interface was implemented as a standalone HTML dashboard rather than relying on a Python web framework. The dashboard displays the inspection summary and embeds the browser-compatible inspection video.

Dashboard component	Displayed value / purpose
Total fastener detections	535
Normal fasteners	456
Defective fasteners	79
Predicted defect ratio	14.8%
Defect breakdown	Deformed 5; Fractured 20; Missing 22; Inverted 22; Displaced 10
Model	YOLO26n
Dataset	RFDD Railway Fastener Defect Dataset
Video	100-frame browser-compatible annotated inspection video

4.13 Complete Implementation Flow



4.14 Implementation Outcome

The implementation successfully connects the major software stages required for the proposed prototype: dataset preparation, deep-learning model training, held-out evaluation, image inference, video inference, automated recording, report generation and dashboard visualization. The final browser-compatible video demonstrates that the inspection output can be presented as an integrated visual monitoring interface.

4.15 Chapter Summary

This chapter documented the actual development work performed for the ITMS prototype. The system was implemented in Python using YOLO26n, PyTorch, OpenCV and supporting data-processing tools, with HTML used for reporting and dashboard visualization. The final implementation produces both quantitative model evaluation and practical inspection artifacts. The next chapter presents the experimental results and detailed performance analysis.

DEVELOPMENT OF INDIGENOUS CONTACTLESS INTEGRATED TRACK MONITORING SYSTEMS (ITMS) FOR TRACK RECORDING

Chapter 5 — Experimental Results and Detailed Analysis

This chapter presents the quantitative evaluation of the final YOLO26n model and the operational outputs produced by the ITMS prototype.

5.1 Experimental Setup

Item	Configuration
Dataset	RFDD Railway Fastener Defect Dataset
Training images	1,050
Validation images	200
Official test images	100
Classes	6 fastener-condition classes
Model	YOLO26n
Final training	40 epochs
Evaluation	Official held-out RFDD test split

5.2 Final Test-Set Performance

The final 40-epoch YOLO26n model was evaluated on the official 100-image RFDD test set. The resulting metrics provide the primary quantitative evidence for the effectiveness of the trained detector.

Metric	Score	Interpretation
Precision	94.1%	High proportion of predicted detections are correct
Recall	88.8%	Most ground-truth fastener instances are detected
mAP@0.50	94.5%	Strong detection performance at IoU 0.50
mAP@0.50:0.95	70.7%	More demanding localization measure across IoU thresholds

5.3 Interpretation of Evaluation Metrics

- **Precision:** A precision of 94.1% indicates that the model's detections were generally reliable, with a relatively small proportion of false-positive predictions.
- **Recall:** A recall of 88.8% indicates that the model detected most of the fasteners represented in the ground-truth test annotations, while some instances were missed.
- **mAP@0.50:** The 94.5% result indicates strong combined localization and classification performance when a 0.50 IoU threshold is used.
- **mAP@0.50:0.95:** The 70.7% result is lower because the metric averages performance over multiple IoU thresholds, requiring increasingly precise localization.

5.4 Comparison of Training Duration

Training duration	mAP@0.50	mAP@0.50:0.95
20 epochs	90.7%	65.8%
40 epochs	94.5%	70.7%
Improvement	+3.8 percentage points	+4.9 percentage points

The 40-epoch model improved both principal mAP measures compared with the 20-epoch model. The 40-epoch weights were therefore retained as the final model used by the inspection scripts and dashboard.

5.5 Class-Wise Test Performance

Class	Images	Precision	Recall	mAP@0.50	mAP@0.50:0.95
Deformed	20	82.2%	80.0%	80.7%	63.5%
Fractured	20	94.0%	78.6%	97.3%	74.3%
Missing	20	91.5%	95.0%	93.8%	59.3%
Inverted	20	98.0%	100.0%	99.5%	81.2%
Normal	100	98.7%	94.9%	99.2%	81.1%
Displaced	20	100.0%	84.1%	96.4%	64.9%

5.6 Class-Wise Analysis

- **Inverted:** This class achieved the strongest overall combination of recall and localization, with 100.0% recall and 81.2% mAP@0.50:0.95.
- **Normal:** The large Normal class achieved 98.7% precision, 94.9% recall and 81.1% mAP@0.50:0.95, indicating strong recognition of normal fasteners.
- **Fractured:** The class achieved 97.3% mAP@0.50, although recall was lower at 78.6%, indicating that some fractured instances were missed.
- **Missing:** Recall was strong at 95.0%, while the stricter mAP@0.50:0.95 was 59.3%, suggesting that localization quality is more challenging for this class.
- **Deformed:** Precision and recall were 82.2% and 80.0%, respectively, with 63.5% mAP@0.50:0.95.
- **Displaced:** Precision reached 100.0% and mAP@0.50 was 96.4%, but recall was 84.1%, meaning some displaced instances were missed.

5.7 Error and Performance Considerations

The difference between the high mAP@0.50 score and the lower mAP@0.50:0.95 score indicates that localization becomes more difficult as the overlap requirement becomes stricter. Class-wise recall also shows that certain visually subtle defect conditions remain harder to detect consistently.

These results are useful because they identify concrete areas for future improvement: better localization, additional representative training examples, stronger augmentation, improved temporal information and evaluation on more diverse railway environments.

5.8 Image-Based Inspection Results

Inspection output	Prediction count
Total fastener detections	584
Normal	484
Deformed	19
Fractured	20
Missing	21
Inverted	20
Displaced	20
Predicted defective detections	100

The image-based recording script processed all 100 official test images and produced 584 model-predicted fastener detections. The five defective classes together accounted for 100 predicted detections. These counts describe the inference output of the recording script and are not ground-truth statistics.

5.9 Video Inspection Results

Video output	Result
Frames processed	100
Total fastener detections	535
Normal predictions	456
Defective predictions	79
Predicted defect ratio	14.8%

The video-monitoring pipeline successfully processed all 100 frames and generated an annotated inspection video. The system produced 535 predicted fastener detections, of which 79 were classified as defective and 456 as normal.

5.10 Dashboard Results

- The dashboard reports 535 total predicted fastener detections.
- It separates 456 normal predictions from 79 defective predictions.
- It displays the five defect categories and their prediction counts.
- It embeds the browser-compatible H.264 inspection video.
- It presents model, dataset, frame count and inspection metadata in one interface.

5.11 Why the Dashboard Counts Differ from the Official Test Metrics

The official test metrics are calculated by comparing model predictions against ground-truth annotations on the 100-image test set. The dashboard counts are simply the number and classes of predictions generated during the separate demonstration inference run. Therefore, the dashboard's 535 detections and 79 defective predictions must not be interpreted as model accuracy, recall or ground-truth defect totals.

5.12 Overall Findings

- The final YOLO26n model achieved strong overall detection performance on the official RFDD test set.
- The 40-epoch model outperformed the 20-epoch model on both mAP measures.
- The model performed particularly strongly on the Inverted and Normal classes under the stricter mAP@0.50:0.95 metric.
- Missing, Deformed and Displaced conditions present comparatively greater localization/detection challenges.
- The prototype successfully transformed model predictions into structured records and visual inspection outputs.
- The video-monitoring pipeline processed all demonstration frames successfully.
- The final HTML dashboard provides a compact interface for presenting inspection statistics and annotated video.

5.13 Validity and Interpretation of Results

The quantitative evaluation is valid for the stated RFDD test configuration and should be reported with the dataset and test-set context. The results demonstrate feasibility of the proposed computer-vision approach, but they do not establish performance on every railway environment or guarantee real-world engineering reliability.

The simulated video demonstrates the software workflow rather than live high-speed railway operation. Consequently, claims about deployment speed, track geometry measurement or operational safety certification are outside the evidence provided by this experiment.

5.14 Chapter Summary

The final YOLO26n model achieved 94.1% precision, 88.8% recall, 94.5% mAP@0.50 and 70.7% mAP@0.50:0.95 on the official RFDD test set. The 40-epoch model improved substantially over the 20-epoch model. The complete prototype also demonstrated automated image and video inspection, structured recording and dashboard reporting. These results establish the technical feasibility of the proposed ITMS-inspired computer-vision workflow while clearly defining its current limitations.

CHAPTER 6

Conclusion, Limitations and Future Scope

6.1 Conclusion

This project presented a software-based prototype for contactless railway track monitoring using computer vision and deep learning. The system was designed around the objective of automatically detecting and classifying railway fasteners from track images and converting the predictions into inspection records and visual monitoring outputs.

The Railway Fastener Defect Dataset (RFDD) was converted into a YOLO-compatible structure while preserving its official train, validation and test separation. A YOLO26n object-detection model was trained for six fastener classes: Deformed, Fractured, Missing, Inverted, Normal and Displaced. The final 40-epoch model achieved 94.1% precision, 88.8% recall, 94.5% mAP@50 and 70.7% mAP@50–95 on the official RFDD test set.

The trained model was then integrated into an end-to-end inspection workflow. The prototype can process test images, generate annotated outputs, record predictions in CSV format, process a simulated inspection video, and present summary information through an HTML inspection report and dashboard. This demonstrates how an AI-based visual inspection component can support automated railway track monitoring.

6.2 Achievement of Objectives

Objective	Status / Outcome
Prepare railway fastener dataset	Completed — RFDD converted to YOLO format with official split preserved.
Train a deep-learning detection model	Completed — YOLO26n trained for 40 epochs.
Detect and classify fastener conditions	Completed — six RFDD classes supported.
Evaluate model performance	Completed — official test metrics and class-wise results obtained.
Create automated inspection records	Completed — CSV records and annotated images generated.
Demonstrate video-based monitoring	Completed — simulated inspection video processed frame by frame.
Provide inspection visualization	Completed — HTML report and standalone dashboard created.

6.3 Limitations

- The developed system is an academic software prototype and is not an engineering-certified replacement for an operational Indian Railways ITMS.
- The experiments use the RFDD dataset. The simulated inspection video is constructed from dataset test images and should not be interpreted as real railway-camera footage.
- The prototype focuses on fastener detection and condition classification. It does not implement the complete set of industrial ITMS measurements such as full track geometry, rail profile/wear measurement, LiDAR-based measurements, ride-quality measurement, or calibrated laser sensing.
- The prediction totals shown in the image/video dashboard are model outputs, not ground-truth defect counts. They should therefore be used for demonstrating system operation rather than as a direct measure of real railway defect prevalence.

- The model was trained and evaluated on a finite dataset. Performance on different camera systems, lighting conditions, track types, motion blur, weather conditions, or unseen railway environments may differ.
- The current implementation was trained using CPU computation, which increases training time and limits rapid experimentation compared with a suitable GPU environment.

6.4 Future Scope

- Integrate real railway inspection video captured using calibrated cameras mounted on a suitable inspection platform.
- Extend the system from fastener defects to additional track components such as rails, sleepers, joints and other infrastructure conditions.
- Add temporal tracking so the same physical fastener can be followed across consecutive video frames and duplicate detections can be reduced.
- Develop severity scoring and maintenance-priority classification based on defect type, confidence, location and recurrence.
- Integrate GPS/chainage information so detected defects can be associated with precise track locations.
- Investigate GPU/accelerated inference and deployment optimization for near-real-time inspection.
- Expand the dashboard to include historical inspections, trend analysis, searchable defect records and maintenance status.
- Evaluate the model on larger and more diverse datasets and conduct field validation before any engineering use.

6.5 Overall Project Outcome

The project successfully demonstrates a complete computer-vision pipeline from railway image data through deep-learning inference to automated inspection recording and visualization. The final prototype establishes a practical foundation for further research into AI-assisted railway inspection while clearly separating academic demonstration from full-scale railway engineering deployment.

6.6 Final Statement

The work shows that modern object-detection methods can be applied effectively to railway fastener inspection and can transform individual visual predictions into structured inspection information. With additional real-world data, calibrated sensing, temporal tracking, location integration and field validation, the prototype could be developed toward a more comprehensive railway inspection support system.

CHAPTER 7

References and Bibliography

7.1 Official Railway and ITMS References

- [1] Press Information Bureau, Government of India. "Indian Railways deploys Indigenous Contactless Integrated Track Monitoring System (ITMS) for Comprehensive Track Inspection and Monitoring." 12 March 2026.
- [2] Press Information Bureau, Government of India. "Indian Railways deploys advanced Integrated Track Monitoring System (ITMS) for comprehensive track inspection." 5 December 2024.
- [3] Research Designs & Standards Organisation (RDSO), Indian Railways. Draft Technical Specification: Integrated Track Monitoring System (ITMS).
- [4] Research Designs & Standards Organisation (RDSO), Indian Railways. Annual Report 2022–23.

7.2 Dataset References

- [5] NIM-NMDC. "RFDD: Railway Fastener Defect Dataset." GitHub repository. National/industrial research dataset for railway fastener defect detection.
- [6] RFDD Dataset. Science Data Bank. DOI: 10.57760/sciencedb.msdc.00071.
- [7] Scientific Data (Nature). "RFDD: Railway Fastener Defect Dataset" / dataset description and methodology. Published 17 July 2026.

7.3 Deep Learning and Object Detection References

- [8] Ultralytics. YOLO documentation and Ultralytics implementation used for object detection and model training.
- [9] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. "You Only Look Once: Unified, Real-Time Object Detection." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [10] Bochkovskiy, A., Wang, C.-Y., and Liao, H.-Y. M. "YOLOv4: Optimal Speed and Accuracy of Object Detection." arXiv, 2020.
- [11] Jocher, G., Chaurasia, A., and Qiu, J. Ultralytics YOLO documentation and software ecosystem for modern object detection.

7.4 Software and Technical References

- [12] Python Software Foundation. Python programming language documentation.
- [13] HDF Group. HDF5 documentation and h5py-compatible hierarchical data format resources.
- [14] OpenCV. Open Source Computer Vision Library documentation, used for image/video processing and video generation.
- [15] FFmpeg. Multimedia framework documentation, used to convert the generated inspection video into browser-compatible H.264 format.

7.5 Project-Generated Artifacts

[16] RFDD YOLO dataset prepared for this project, preserving the official RFDD train/validation/test split.

[17] YOLO26n model trained for 40 epochs and evaluated on the official RFDD test set.

[18] ITMS automated track inspection CSV records and annotated inspection images generated by the project prototype.

[19] Simulated track inspection video and frame-by-frame video inspection records generated from RFDD test images.

[20] Standalone HTML ITMS inspection report and dashboard developed as project visualization outputs.

7.6 Note on Referencing

The references above support the railway ITMS background, dataset, machine-learning methodology, and software technologies used in this academic prototype. Project-generated artifacts are listed separately because they are outputs produced during the implementation rather than external sources.